MEMORANDUM

To:	Professor Brendon Anderson, Mechanical Engineering, California Polytechnic State University, bga@calpoly.edu
From:	Gabriel Choi, Carmen Alaichamy
Date:	May 13, 2026
Subject:	Lab 4: Tracking Control

Introduction

The objective of this lab was to implement a trapezoidal velocity profile for tracking control. The goal is to have the PI control match the given velocity profile in Figure 1. Finally, we will analyze the performance of the controller with a ramp input.

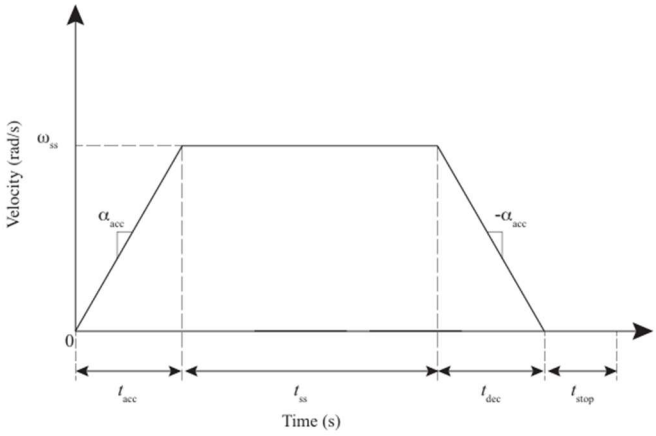


Figure 1. Given velocity profile.

Methodology

The experiment was conducted using an STM32 microcontroller connected to a DC motor and rotary encoder. We completed the file profile.py shown in Appendix A to create the velocity profile generator. We completed the generator function trapz_profile_gen() using equations based on the velocity profile graph. Then the method trapz_profile_length() computes the number of points needed. Running profile.py yields our desired velocity profile shown in Figure 2. The class file was then uploaded to the microcontroller using Thonny.

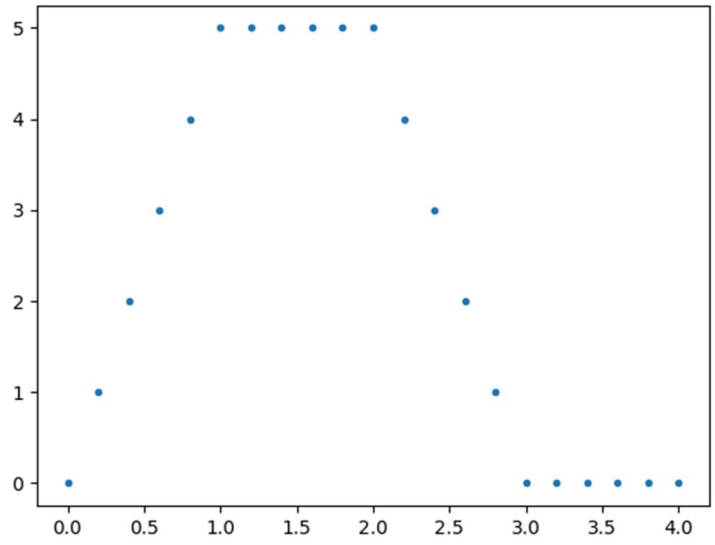


Figure 2. Profile.py velocity profile.

The main.py file is shown in Appendix B, it was completed to set the setpoint, run the controller, apply the correct voltage and collect data into queues. The main.py was tested and uploaded to the microcontroller.

To graph, the Jupyter Notebook file was used. This is shown in Appendix C, including the plots created.

Results

The following plots in Figure 3-7 shows the overlaid response plots of the ideal speed profile and the actual response. Figure 3 shows the gains from Lab 3b. Figure 4 and 5 modify these gains to

get the optimal response and an overshooting response, respectively. Figure 6 and 7 decrease acceleration to 0.25 seconds and show varying gains.

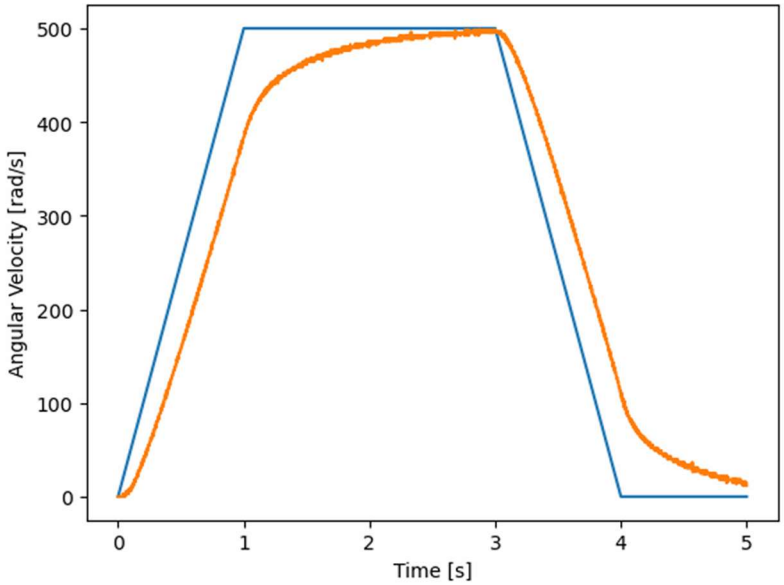


Figure 3. Overlaid response plot of ***original*** speed profile and actual response, ***with controller gains from Lab 3B***. (Parameters: Steady state velocity = 500 rad/s, steady state time = 2 s, stop time of 1, acceleration time = 1 s, Kp = 0.19, KI = 0.4)

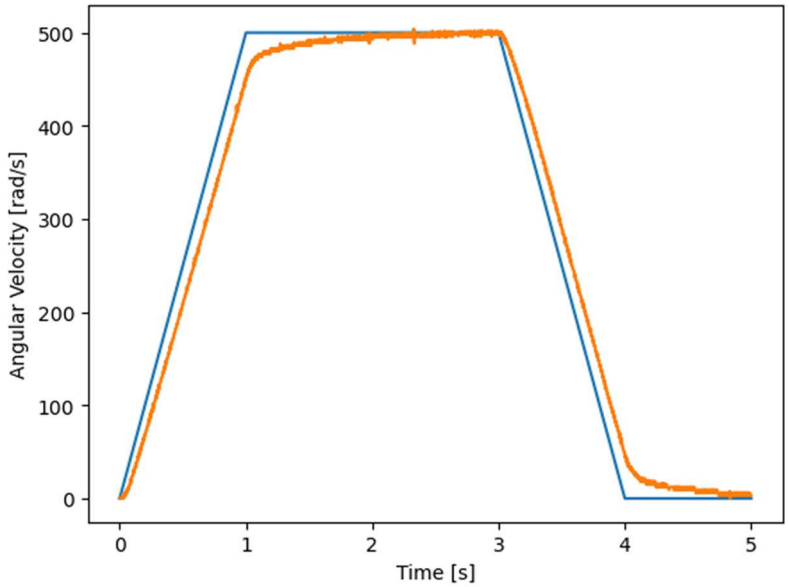


Figure 4. Overlaid response plot of ***optimal*** speed profile and actual response, ***with modified controller gains***. (Parameters: Steady state velocity = 500 rad/s, steady state time = 2 s, stop time of 1, acceleration time = 1 s, Kp = 0.5, KI = 1.0)

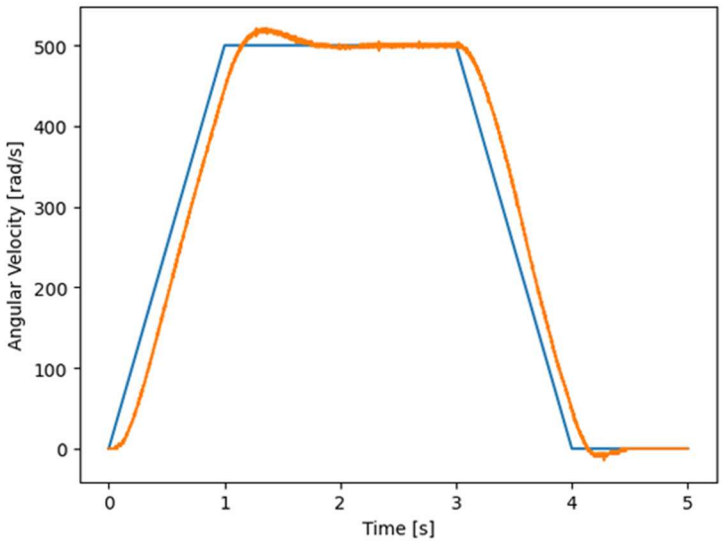


Figure 5. Overlaid response plot of ***overshooting*** speed profile and actual response, ***with modified controller gains***. (Parameters: Steady state velocity = 500 rad/s, steady state time = 2 s, stop time of 1, acceleration time = 1 s, Kp = 0.1, KI = 1)

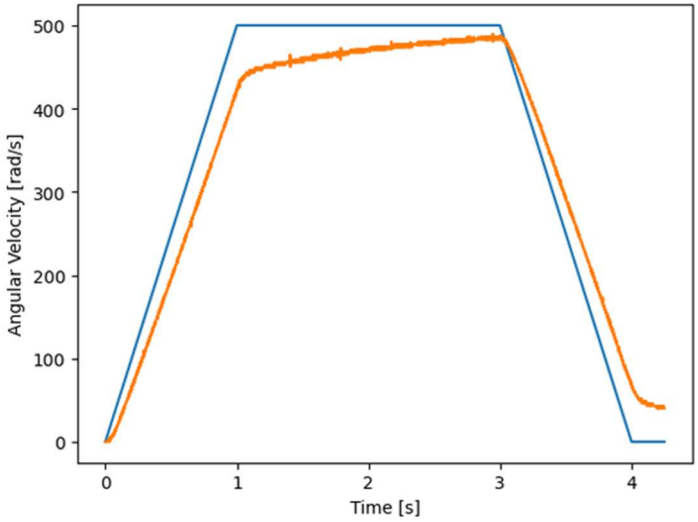


Figure 6. Overlaid response plot of speed profile and actual response, ***with 0.25 acceleration and modified controller gains***. (Parameters: Steady state velocity = 500 rad/s, steady state time = 2 s, stop time of 1, acceleration time = 0.25 s, Kp = 0.5, KI = 0.4)

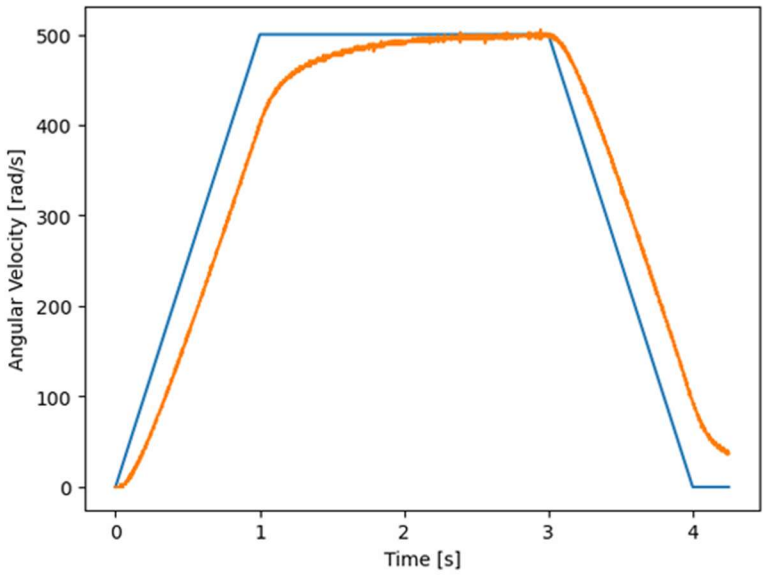


Figure 7. Overlaid response plot of speed profile and actual response, ***with 0.25 acceleration and modified controller gains***. (Parameters: Steady state velocity = 500 rad/s, steady state time = 2 s, stop time of 1, acceleration time = 0.25 s, Kp = 0.2, KI = 0.5)

Discussion

Explain how PI gains affect the ramp response. Is steady state error eliminated during the acceleration and deceleration of the motor?

The PI gains affect ramp response with each Kp and KI contributing differently. Kp acts to quickly changing ramp response to decrease error. KI integrates error over time to eliminate steady state error.

Between Figure 6 and 7, KI is kept relatively constant while Kp was decreased from 0.5 to 0.2. The tracking in Figure 7 is clearly worse, with less Kp leading to slower response.

Steady state error is eliminated best in Figure 4, where KI was increased to 1 from the previous 0.4 in Figure 3. Figure 3 reaches steady state with more error because there is less integral controller to recognize and fix the error.

Steady state error is eliminated most in Figure 4 because of the optimal balance between Kp and KI controllers. While the steady state error becomes completely eliminated when the motor is at a constant speed, it is still not completely eliminated while the motor is accelerating or decelerating. The motor changing seems to make the controllers too slow to fully account for the error, or predict where the controller should operate.

Appendix A: profile.py

```
def trapz_profile_length(omega_ss, t_acc, t_ss, t_stop, delta_t):
    t_total = 2 * t_acc + t_ss + t_stop
    return int(t_total // delta_t) + 2

def trapz_profile_gen(omega_ss, t_acc, t_ss, t_stop, delta_t):
    # t_acc, t_ss and t_stop must be a multiple of delta_t.
    for i in range(trapz_profile_length(
        omega_ss, t_acc, t_ss, t_stop, delta_t)):
        # WRITE YOUR CODE HERE to calculate t and omega, and generate the
        # required profile
        t = i * delta_t
        if 0 <= t < t_acc:
            omega = (omega_ss * t)/t_acc
        if t_acc <= t < t_acc + t_ss:
            omega = omega_ss
        if t_acc + t_ss <= t < 2*t_acc + t_ss:
            omega = omega_ss - (omega_ss/t_acc)*(t -t_acc - t_ss)
        if 2*t_acc + t_ss <= t < 2*t_acc + t_ss + t_stop:
                omega = 0
        yield t, omega

if __name__ == '__main__':
    import numpy as np
    from matplotlib import pyplot as plt

    profile = np.array(list(trapz_profile_gen(5, 1, 1, 1, 0.2)))
    ts = profile[:,0]
    omegas = profile[:,1]

    plt.plot(ts, omegas, '.')
    plt.show()

```

Appendix B: main.py

```
from profile import *
import cqueue
import motor
import encoder
import pyb
import utime
import controller

# This code creates motor driver and encoder objects (same as always)
driver = motor.MotorDriver()
driver.motorA.enable()
enc = encoder.Encoder(4, pyb.Pin.cpu.B6, pyb.Pin.cpu.B7)
enc.zero()

# Collects all the parameters of the sine sweep, either from user input
# or from a jupyter notebook
while True:
    try:
        omega_ss = float(input("Steady State Velocity [rad/s]: "))
        t_ss = float(input("Steady State Velocity Time [s]: "))
        t_acc = float(input("Acceleration/Deceleration Time [s]: "))
        t_stop = float(input("Stopped Time [s]: "))
        kp = float(input("Proportional Gain [%/rad/s]: "))
        ki = float(input("Integral Gain [%/rad]: "))
    except ValueError:
        print("Invalid input")
        continue
    else:
        break

period_us = 2000
delta_t = period_us / 1e6

con = controller.PIController(kp, ki, 100, delta_t, 0)

# Queues to store data (faster than arrays, must specify size)
N = trapz_profile_length(omega_ss, t_acc, t_ss, t_stop, delta_t)
t_queue = cqueue.FloatQueue(N)
omega_sp_queue = cqueue.FloatQueue(N)
omega_meas_queue = cqueue.FloatQueue(N)

# Used to keep the main loop running at the right period
next_time_us = utime.ticks_us() + period_us

sweep_profile = trapz_profile_gen(omega_ss, t_acc, t_ss, t_stop, delta_t)
# Main loop
for t, omega_sp in sweep_profile:
    # Read the velocity of the encoder, in radians per second
    omega_meas = enc.get_velocity_rad()

    # WRITE YOUR CODE HERE to set the setpoint to omega_sp, generated by profile
    con.set_setpoint(omega_sp)
    # WRITE YOUR CODE HERE to run the controller using the measured velocity
    volt_percent = con.run(omega_meas)
    # WRITE YOUR CODE HERE to set the percent voltage on the motor
    driver.motorA.set_voltage_percent(volt_percent)
    # WRITE YOUR CODE HERE to put the data in the queues. t_queue is an example
    t_queue.put(t)
    omega_meas_queue.put(omega_meas)
    omega_sp_queue.put(omega_sp)

    # Loop timing, DO NOT write any code past this point in the for loop
    # Wait until it is time to run the next point. pass means do nothing
    while utime.ticks_diff(utime.ticks_us(), next_time_us) < 0:
```

```
        pass
    # Calculate the next time to run the loop
    next_time_us = utime.ticks_add(next_time_us, period_us)

# Shut off the motor
driver.motorA.disable()

# Print the data gathered
while t_queue.any():
    print(t_queue.get(), omega_sp_queue.get(), omega_meas_queue.get(), sep=',')
# Marker for data end, detected by Jupyter Notebook
print('END')

```

Appendix C: Jupyter

In [97]:
```
from serial import Serial
import numpy as np
from matplotlib import pyplot as plt
```

In [98]:
```
def plot_profile(filename):
    data = np.genfromtxt(filename, delimiter=',')
    t = data[:,0]
    v_sp = data[:,1]
    v_meas = data[:,2]

    plt.figure()
    plt.plot(t, v_sp)
    plt.plot(t, v_meas)
    plt.xlabel('Time [s]')
    plt.ylabel('Angular Velocity [rad/s]')
    plt.show()
```

In [99]:
```
def run_profile(omega_ss, t_ss, t_acc, t_stop, kp, ki, filename):
    # Uncomment this line and give the name of the serial port which the Nucleo
    # is connected to, as a string. This should be something like
    # 'COMx' for Windows and
    # '/dev/tty.usbmodemXXXX' for Mac/Linux
    # Windows: Open Device Manager and look under "Ports (COM & LPT)" for a
    # "USB Serial Device"
    # Mac/Linux: Open terminal and run "ls /dev/tty.*" to find the port matching th
    # format above.
    serport = 'COM8'

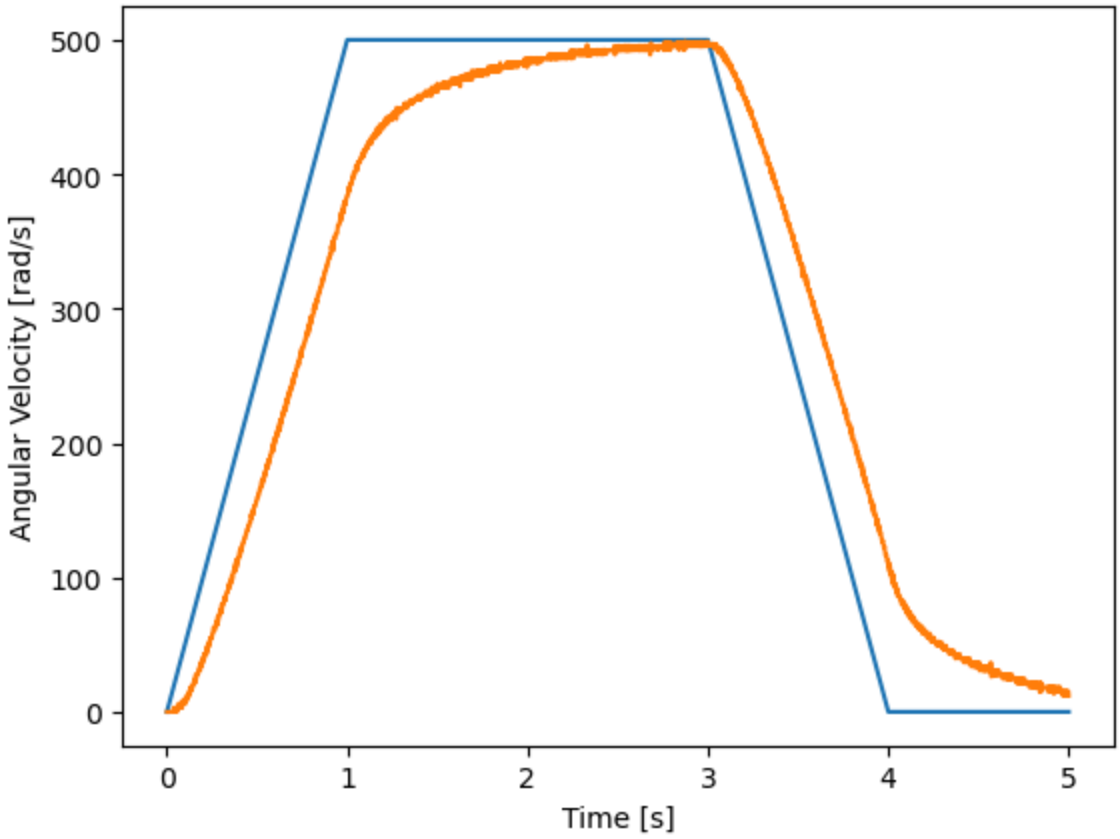
    # Open the serial port, and give it the name ser.
    with Serial(serport, baudrate=115200, timeout=5) as ser:
        # Restart the microcontroller
        ser.write(b'\x02\x04')
        # Write the parameters to the microcontroller
        ser.write((str(omega_ss) + '\r\n').encode())
        ser.write((str(t_ss) + '\r\n').encode())
        ser.write((str(t_acc) + '\r\n').encode())
        ser.write((str(t_stop) + '\r\n').encode())
        ser.write((str(kp) + '\r\n').encode())
        ser.write((str(ki) + '\r\n').encode())

        # Open the data file for writing, and give it the name datafile
        with open(filename, 'w') as f:
            while True:
                # Read the current line and decode from bytes to a string
                line = ser.readline().decode()
                # Print the line so we can see what the microcontroller is doing
#                 print(line, end='')
                # If the line starts with END, then we know the data is complete
                if line[:3] == 'END':
                    break
                # If the line has a comma, it is a line of CSV data.
                elif ',' in line:
                    f.write(line.strip() + '\n')
```

```
        plot_profile(filename)
```

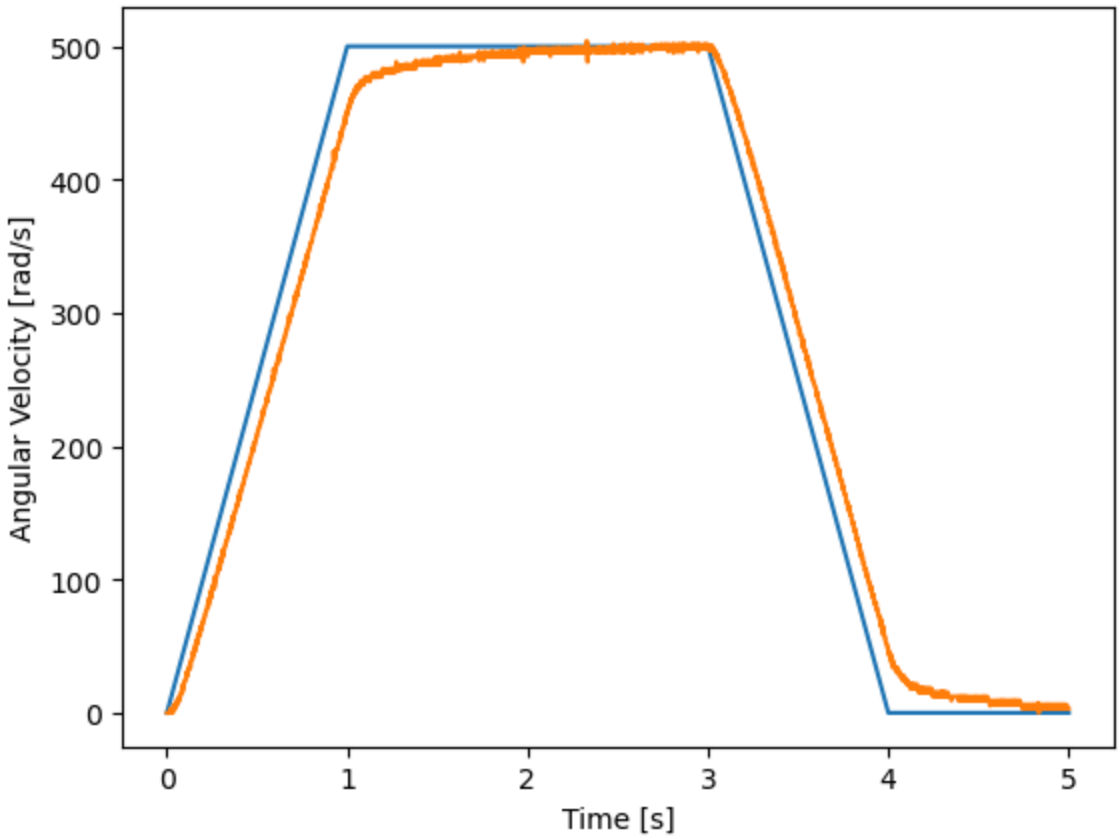
In [100…

```
run_profile(500, 2, 1, 1, 0.19, 0.4, 'data1.csv')
```

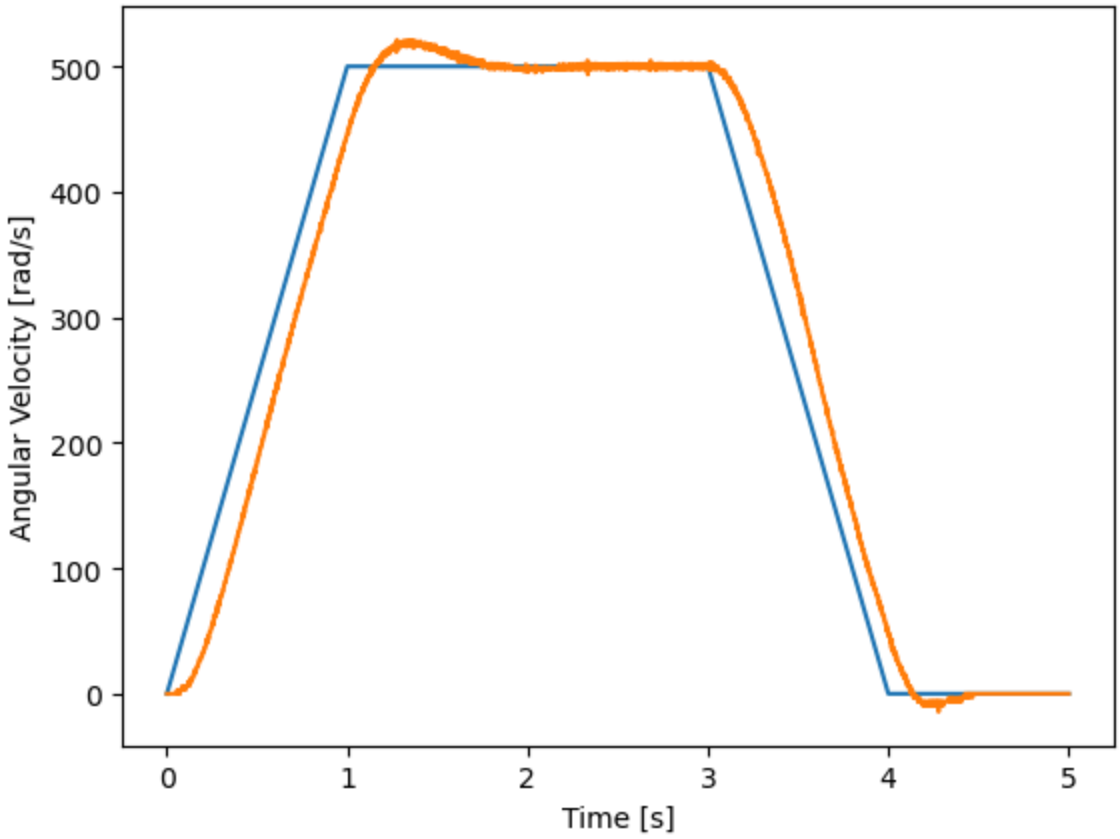


In [101…

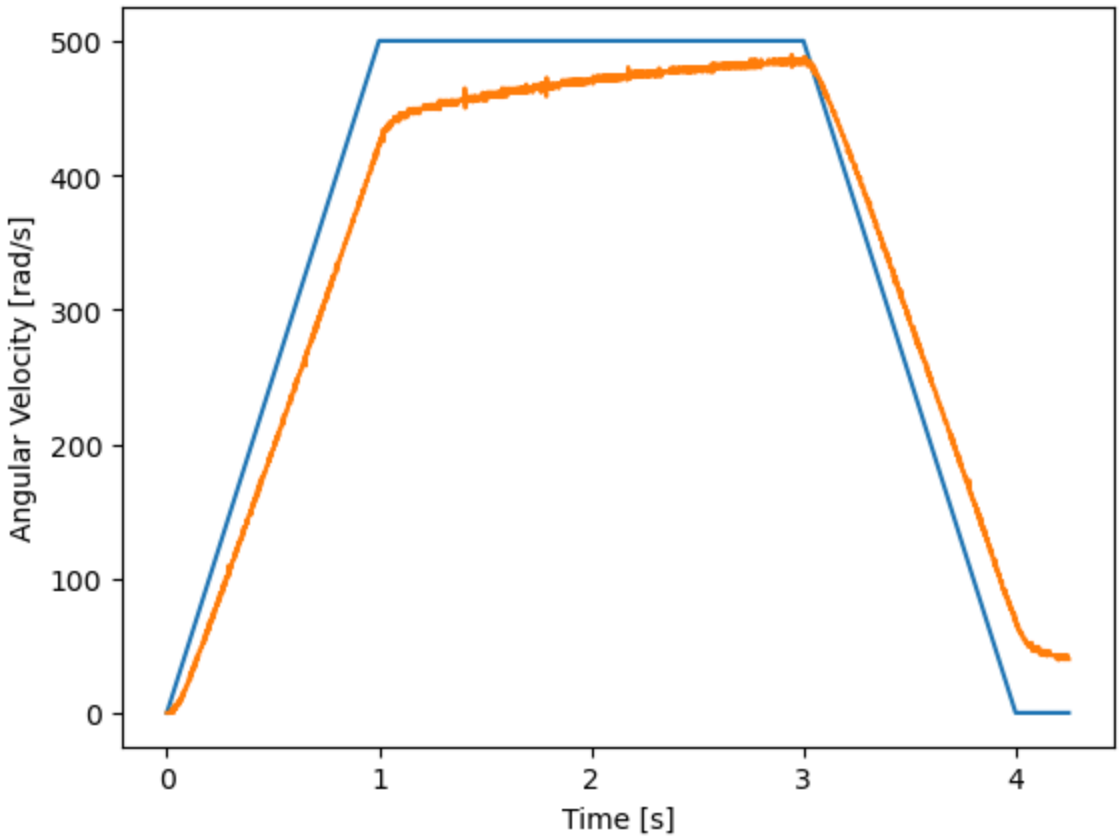
```
run_profile(500, 2, 1, 1, 0.5, 1, 'data2.csv')
```



In [102…
```
run_profile(500, 2, 1, 1, 0.1, 1, 'data3.csv')
```

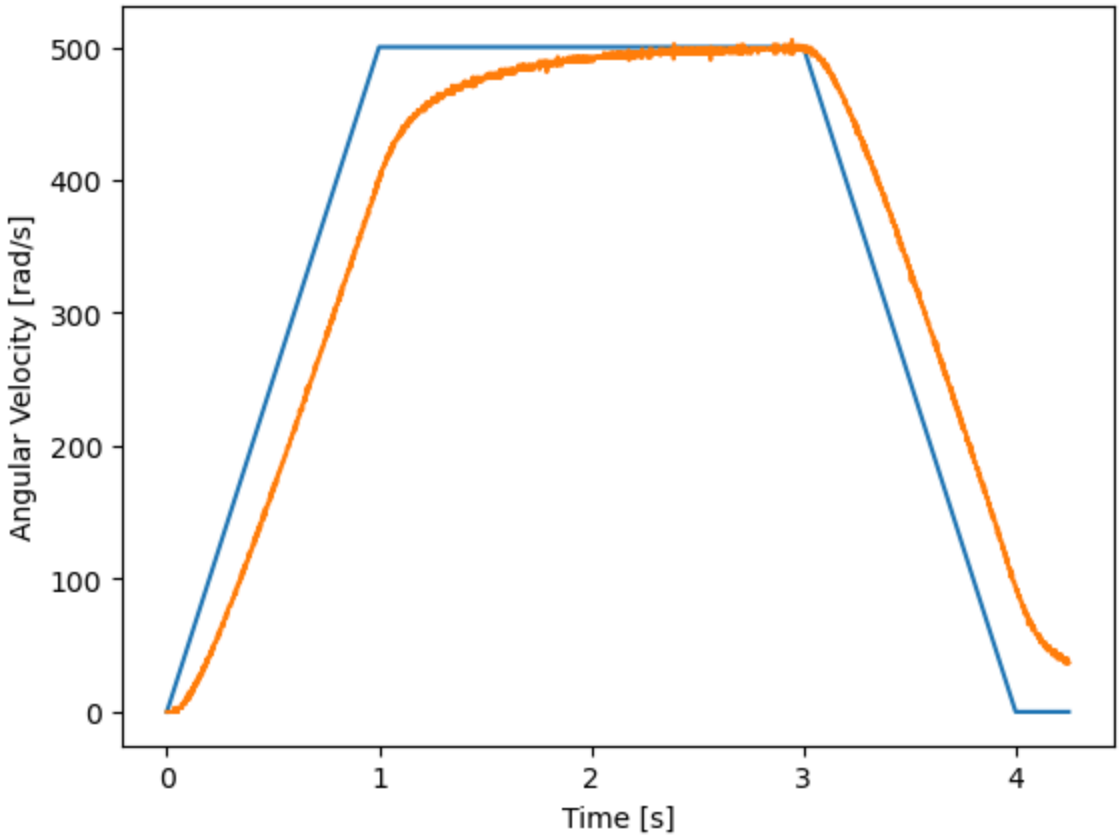


In [103…
```
run_profile(500, 2, 1, 0.25, 0.5, 0.4, 'data4.csv')
```



In [104…

```
run_profile(500, 2, 1, 0.25, 0.2, 0.5, 'data5.csv')
```



In [ ]: